

Les calculatrices, téléphones, tablettes, montres connectées et tous les appareils électroniques de communication ou de stockage, ainsi que les documents sont interdits.

Exercice 1 (4 points)

Code du programme

```
n = 6
s = 0
for i in range(1, n+1):
    if i % 2 == 0:
        s = s + i
    else:
        s = s - i
print(s)
```

Questions

- Quel est le résultat imprimé à l'écran lorsque le programme est exécuté?
- Proposez une fonction `operation_nombre(n)` qui applique le même programme pour tout entier $n \geq 1$ et retourne le résultat final.
- Quelle est la valeur retournée par l'appel `operation_nombre(10)`?

Exercice 2 (10 points)

Vous gérez les présences d'étudiants dans plusieurs matières. Chaque matière est un dictionnaire : les clés sont les matricules et les valeurs indiquent "signé" ou "non signé".

```
matiere1 = { "I19240": "signé", "I20360": "non signé", "I21540": "signé", ..... }
```

- Ajouter l'étudiant "I25001" comme "signé" à `matiere1`.
- Afficher le nombre total d'étudiants dans `matiere1`.
- Afficher la liste des étudiants absents dans `matiere1`.
- Écrire la fonction `comparer(matiereA, matiereB)` qui prend deux dictionnaires (comme `matiere1`) et retourne la liste des étudiants qui sont absents dans `matiereA` mais présents dans `matiereB`.
- Écrire la fonction `stats_exam(matières)` qui prend en paramètre une liste de dictionnaires représentant les présences dans plusieurs matières et retourne le taux global de présence (en %) :

$$\text{Taux global de présence (\%)} = \left(\frac{\text{Nombre total de 'signé'}}{\text{Nombre total d'étudiants}} \right) \times 100$$

- Créer la classe `FeuillePresence` avec les attribut `nom_matiere` et dictionnaire de présences.
- Créer et tester la méthode `ajouter_etudiant(self, matricule, statut)` : ajoute un étudiant.

Exercice 3 (8 points)

- Écrire la fonction `minutes_avant_minuit(h, m)` qui prend l'heure actuelle (en heures et minutes) et retourne le nombre de minutes restantes avant minuit. Gérer les cas limites (23h59, 0h0...).
- Écrire la fonction `somme_impairs(n, m)` qui renvoie la somme des entiers impairs strictement compris entre n et m (exclus). Gérer le cas où $n > m$.
- Analyser le code suivant et donner les valeurs de `D1`, `D2` et `D3` :

```
Valeurs = [i for i in range(1, 20) if i % 2 == 0 or i % 3 == 0]
D1 = Valeurs[5]
D2 = Valeurs[-3:]
D3 = [x**2 for x in Valeurs if x % 4 == 0]
```

- Écrire la fonction réursive `voyelles(c)` qui prend une chaîne de caractères et retourne le nombre de voyelles qu'elle contient. Les voyelles sont : a, e, i, o, u, y (en minuscules ou majuscules).